Data & AI Builders Conference

Building modern data foundations for intelligent solutions together

Query Store and SSMS Copilot

Who Is the Fairest in the Land?

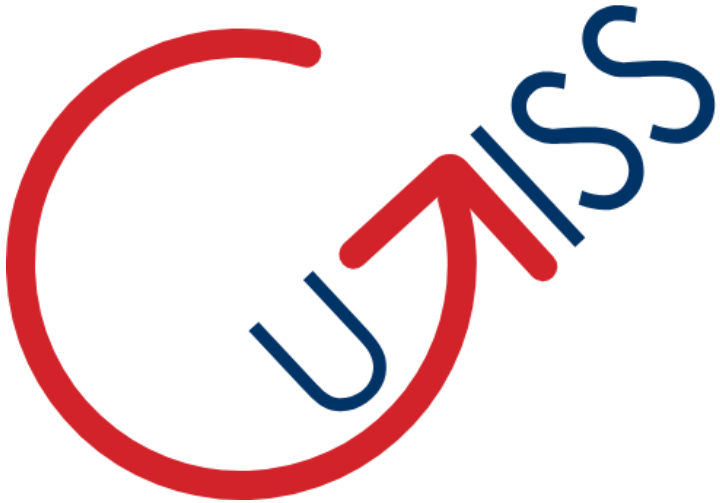
From Query Store evidence to guided and autonomous troubleshooting with SSMS Copilot, Agent Mode and Agent Skills



Sponsors



Collaborators





whoami



Marzio Da Ros

support Engineer @Microsoft

- Supporting SQL Server Core
- 4 Years @ Microsoft
- Worked previously as DBA in other companies for more than 25 years, managing SQL Server since SQL Server 2000 version.

What you will learn

Three outcomes for today's session

1 · Read the evidence

Use Query Store to identify the dominant performance resource, the main contributing query, and how its behavior changes over time.

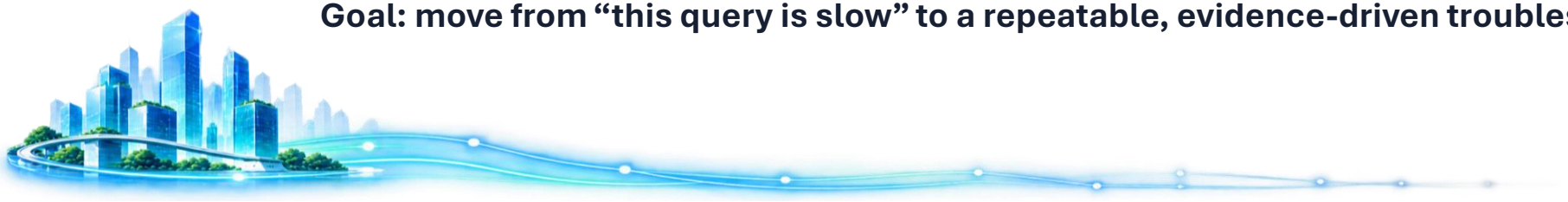
2 · Diagnose the change

Correlate plans, CPU, duration, logical reads, selectivity, statistics, and indexes to build a root-cause hypothesis from evidence.

3 · Automate the method

Use GitHub Copilot Agent Mode and reusable Agent Skills to standardize SQL performance troubleshooting across SQL Server and Azure SQL.

Goal: move from "this query is slow" to a repeatable, evidence-driven troubleshooting workflow.



Lodestar

Agenda

One troubleshooting story — three Copilot experiences

ASK MODE

Guided investigation

You ask, Copilot interprets the current evidence.

AGENT MODE

Autonomous investigation

Copilot executes approved read-only diagnostics and iterates.

AGENT SKILL

Reusable methodology

Encode the runbook once and reuse it from SSMS / GitHub Copilot.

Troubleshooting storyline

1. Symptom
2. Dominant metric
3. Top query
4. Plans & timeline
5. Same-plan degradation
6. Selectivity / stats / indexes
7. Remediation
8. Validation



Lodestar

The starting point

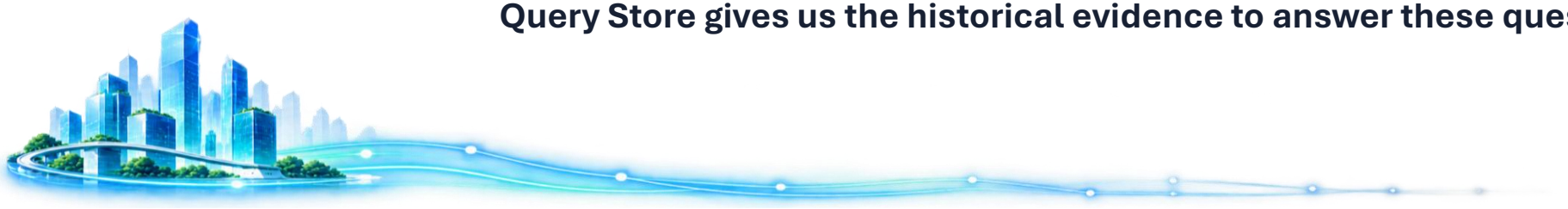
"SQL is slow." That is a symptom — not a diagnosis.

The first question is not "which plan is bad?"

The first question is: what resource is driving the impact?

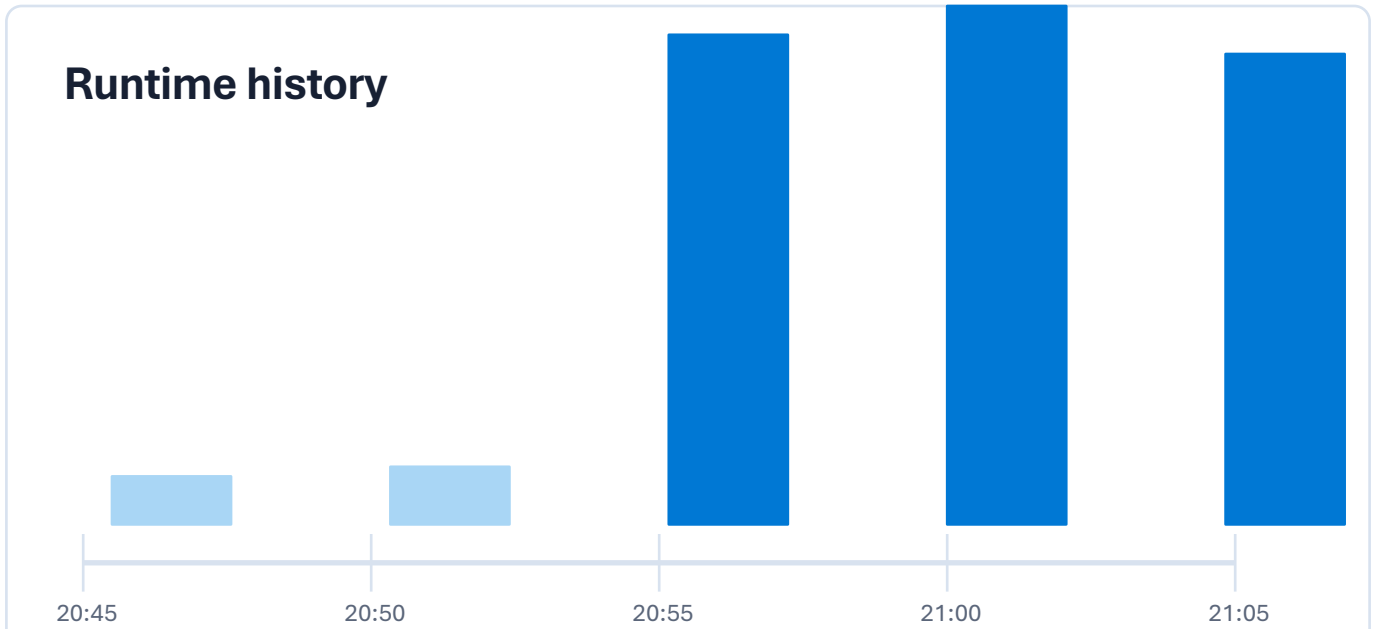
CPU	How much processor time?
Duration	How long does it take?
Logical reads	How much data is touched?
Executions	How often does it happen?
Waits	Where does the query spend time?

Query Store gives us the historical evidence to answer these questions.



Query Store = the evidence layer

History lets us compare what changed, not just what is happening now



same query • same plan can still behave very differently

Query

Query text · Query ID · object name

Plans

Plan IDs · first/last seen · plan history

Runtime

CPU · duration · logical reads · executions · waits

Our demo lab



Powershell workload

Random @N values
Repeated executions

dbo.GiveNotes

WHERE ID BETWEEN @N …
GROUP BY Name

dbo.Notes

Rows grow
Distribution changes

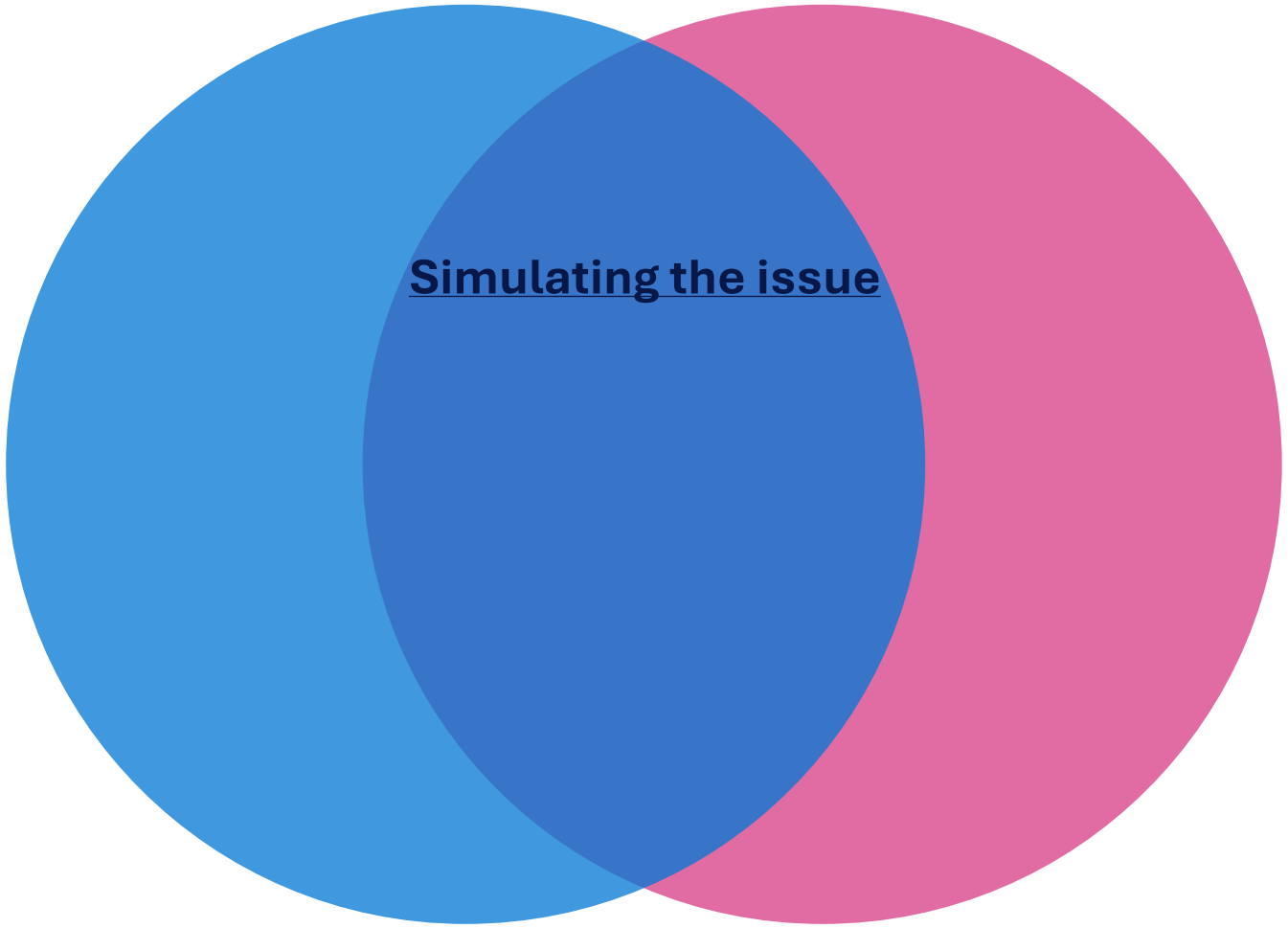
Query Store + Copilot

Observe · compare · explain · validate

We intentionally avoid giving Copilot the answer: it must discover the query, Query ID, all Plan IDs, the timeline, and the most likely explanation from evidence.



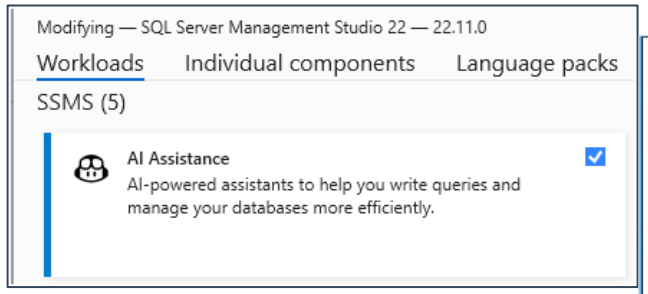
Live demo

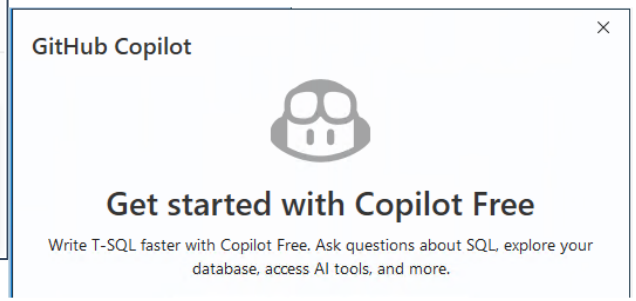




Preparing the environment (1)

Prerequisites:
- SSMS **22.7 or later**
- The **AI Assistance** workload installed
- GitHub Copilot **access** in SSMS
- GitHub Copilot Chat running in **Agent** mode





Mode 1 — Ask

A guided conversation with Query Store evidence

1. **What resource dominates?**
2. **Which query contributes most?**
3. **How does it behave over time?**
4. **What changed while the same plan was reused?**

What we can learn

Ask mode is excellent for interpreting the current grid and walking through evidence step by step.

But in our session it could not execute arbitrary new ad-hoc Query Store T-SQL from chat.

That is where Agent Mode changes the experience.



Live demo


Ask SSMS
1. Find the problem
2. Watch the Agent investigate
3. Re-run with the reusable skill

Mode 2 — Agent

Give Copilot the goal, not the individual steps

High-level goal — “Investigate the SQL performance problem.”

Plan the investigation — Choose the next read-only diagnostic.

Execute with approval — Use SSMS sql-tools and the current connection.

Read the result — Update the working hypothesis from evidence.

Iterate — Continue until diagnosis or user input is needed.

Agent Mode in SSMS

sql-tools | read-only diagnostics | approval loop

Agent Mode can execute queries, inspect the result, and decide the next diagnostic step — while the SQL execution still runs under the permissions of the active database context.

This is the workflow we wanted from the beginning.



Lodestar

Live demo


Agent SSMS
1. Find the problem
2. Watch the Agent investigate
3. Re-run with the reusable skill

Mode 3 — Agent Skill

Turn a good troubleshooting conversation into a reusable operating procedure

```
.github/
└── skills/
    └── sql-performance-investigator/
        ├── SKILL.md
        └── references/
            ├── workflow.md
            └── query-store-rules.md
```

One skill, many entry points

SSMS Agent

Query Store + sql-tools

GitHub Copilot CLI

Skill + MCP / SQL tool

Visual Studio / IDE

Same method, new surface

Team standard

Evidence → diagnosis → validation

Guardrails: read-only discovery first • no hard-coded Query IDs / Plan IDs • separate evidence from hypothesis • propose remediation, do not silently apply it



Lodestar

Run the investigation in SSMS Agent Mode

One high-level goal; SSMS Agent Mode executes approved read-only diagnostics and keeps the investigation state

1. **Open Copilot Chat**
   View → GitHub Copilot Chat
2. **Switch to Agent**
   Choose Agent instead of Ask
3. **Enable SQL tools**
   Tools → verify sql-tools is enabled
4. **Enable the skill**
   Skills → sql-performance-investigator
5. **Start the investigation**
   Connect to the target database and paste the high-level prompt
6. **Approve diagnostics**
   Let the Agent query, read the result, and choose the next diagnostic

```
Use the sql-performance-investigator skill.

Investigate the SQL performance problem in the
current database using Query Store.

Automatically discover the dominant metric, target
query, Query ID, all Plan IDs, runtime history, and
any same-plan degradation.

Read-only diagnostics only. Do not apply remediation.
```

Key behavior: no manually typed Query IDs or Plan IDs.



Lodestar

Preparing the environment (2)

Skills:

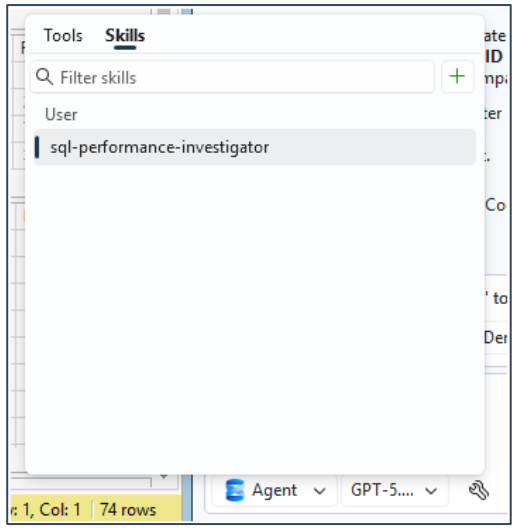
- **Location of Copilot skills files used in this demo:**

  “%USERPROFILE%\.copilot\skills\sql-performance-investigator\”

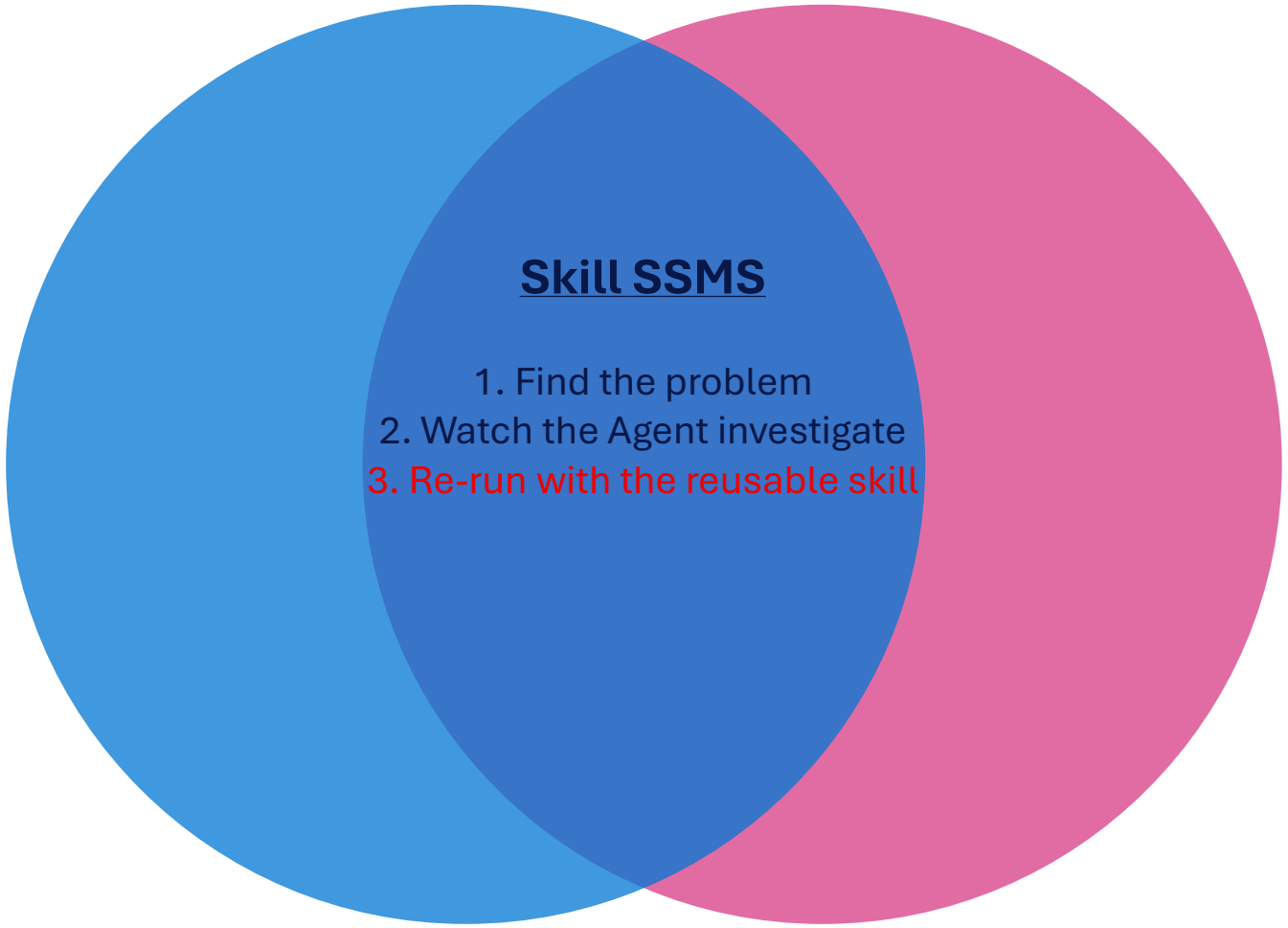
- Useful link related to skills:
  https://learn.microsoft.com/en-us/ssms/github-copilot/agent-skills

- Skill related files used in this demo:

```
FullName                                                                                  Size
--------                                                                                ------
C:\Users\testuser\.copilot\skills\sql-performance-investigator\SKILL.md                   6872
C:\Users\testuser\.copilot\skills\sql-performance-investigator\references\query-store-rules.md   2874
C:\Users\testuser\.copilot\skills\sql-performance-investigator\references\workflow.md     5400
```



Live demo


Skill SSMS
1. Find the problem
2. Watch the Agent investigate
3. Re-run with the reusable skill

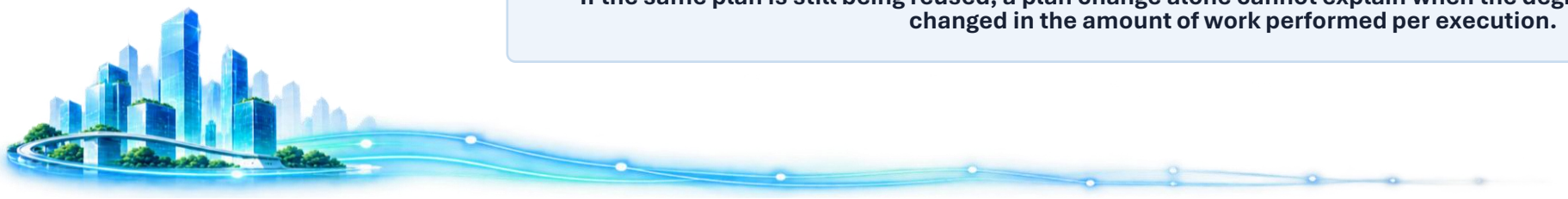
The key diagnostic moment

Same Plan ID. Radically different runtime cost.

Example captured during the demo workload

Metric	Healthy interval	Degraded interval	Change
Avg CPU / exec	7.69 ms	1,443.19 ms	**~188×**
Avg duration / exec	12.39 ms	2,473.68 ms	**~200×**
Avg logical reads	68.39	12,233.10	**~179×**

If the same plan is still being reused, a plan change alone cannot explain when the degradation started. Something changed in the amount of work performed per execution.



From evidence to root cause

Same plan

Plan identity did not change

More reads

Much more work per execution

Selectivity

Predicate can touch very different row counts

Data distribution

How values are actually distributed

Statistics

What the optimizer believes

Indexes

Which access paths actually exist

The strongest hypothesis becomes a validation target — not a conclusion.

```
Confirmed evidence → inference → hypothesis → read-only validation → remediation
```



Lodestar

A concrete remediation candidate

Only after the evidence supports the access-path problem

```
CREATE NONCLUSTERED INDEX
NOTES_IX1
ON dbo.Notes (ID)
INCLUDE (Name);
```

Candidate, not victory: the change still has to earn its place with before/after Query Store evidence.

Why ID is the key

The predicate is ID < @N. The leading key supports a range access path.

Why INCLUDE(Name)

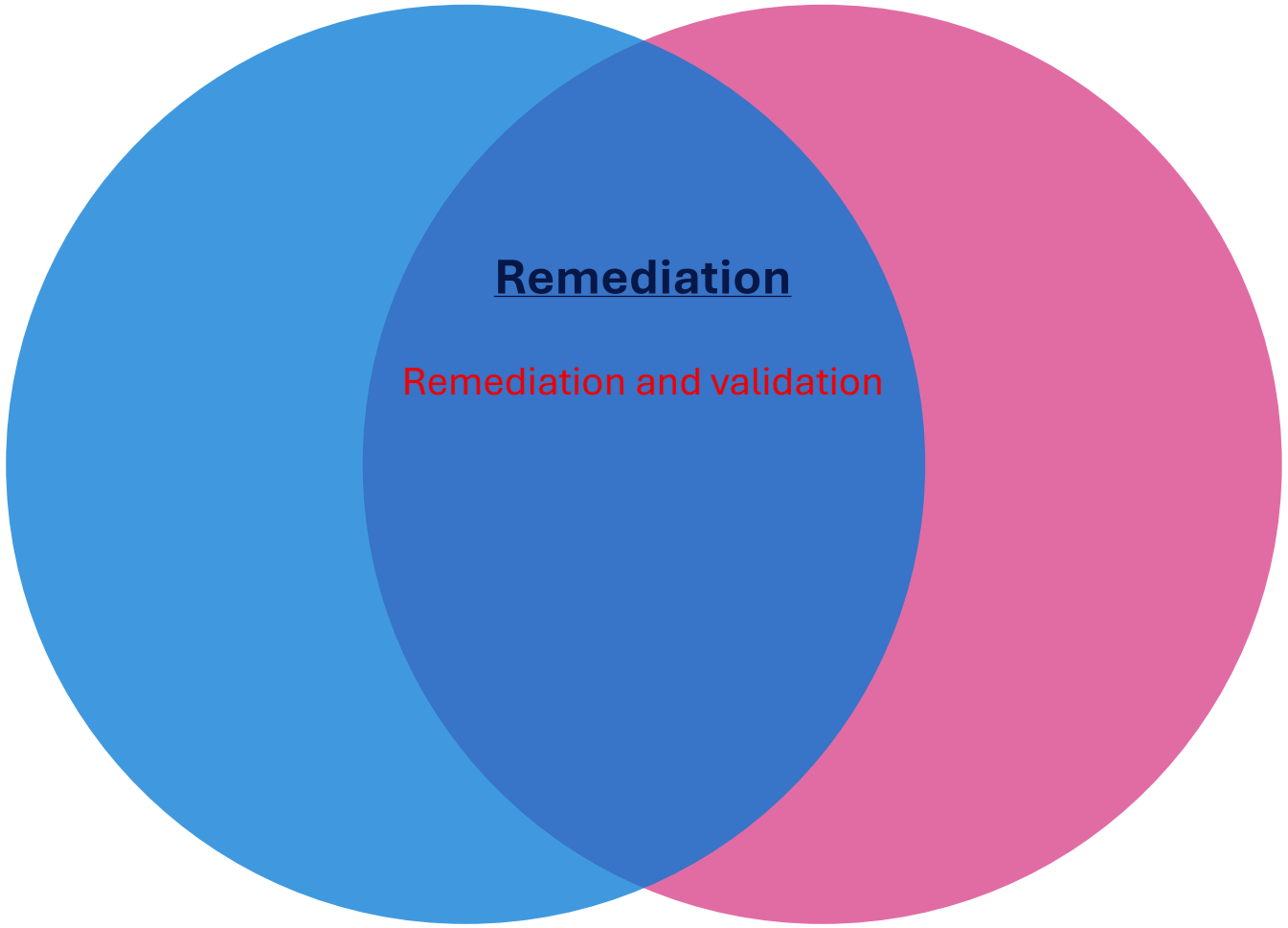
Name is required for the aggregation/output, but it does not need to widen the key.

Statistics caveat

An index creates statistics on its key. INCLUDE columns do not automatically receive a histogram; do not drop auto stats without proving redundancy.



Live demo



The demo storyline — from symptom to a proven fix

What happens during the investigation, and what each piece of evidence tells us

1 Symptom

"SQL is slow."
Start with impact, not a favorite hypothesis.

2 Dominant resource

Query Store ranks CPU, duration, logical reads and execution volume.

3 Main query

Discover dbo.GiveNotes and its Query ID dynamically — never hard-code it.

4 Plans + timeline

Collect every Plan ID and runtime interval. Reference run: 4 plans; IDs vary every run.

5 Critical clue

Same plan: CPU 7.69 → 1,443.19 ms; duration 12.39 → 2,473.68 ms; reads 68.39 → 12,233.10.

6 Explain extra work

WHERE ID < @N changes selectivity. The same plan can process radically different row counts.

7 Validate metadata

Inspect distribution, statistics and access paths. Clustered PK is on id2, not ID.

8 Candidate fix

CREATE INDEX Notes(ID) INCLUDE(Name) — then re-run an equivalent workload.

9 Prove it

Use Query Store before/after: lower reads, CPU, duration and variability; no hidden regression.



Three takeaways

What to remember after the session

1 Query Store tells the story

The useful question is not only “which query is expensive?” but “how did the query and its plans behave over time?”

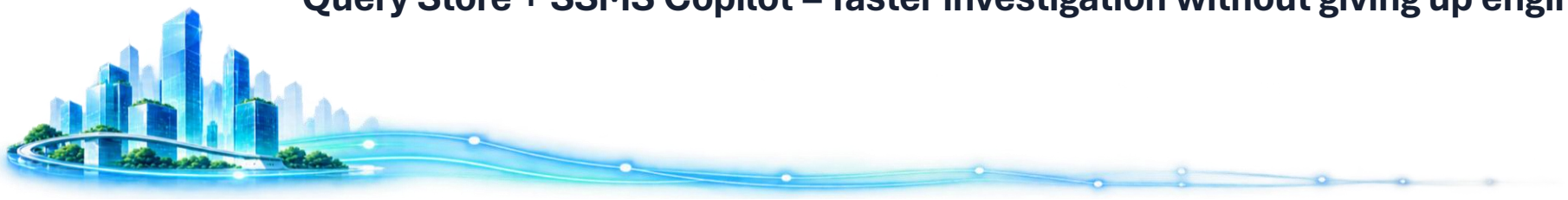
2 Evidence before explanation

Same-plan degradation is a powerful clue. Separate confirmed evidence from inference before naming a root cause.

3 Agents make the method reusable

Agent Mode can run the investigation. Agent Skills turn the support methodology into a repeatable team asset.

Query Store + SSMS Copilot = faster investigation without giving up engineering discipline.



Lodestar

Q&A

Questions, scenarios, and “what would you investigate next?”



Lodestar

Data & AI Builders Conference

Building modern data foundations for intelligent solutions together

THANK YOU !!



Marzio Da Ros

Support Engineer SQL Server @Microsoft

/marzio-da-ros-58718bb

marziodaros@microsoft.com